

Nettverksprogrammering

Virtualisering

Ole C. Eidheim

February 17, 2026

Department of Computer Science

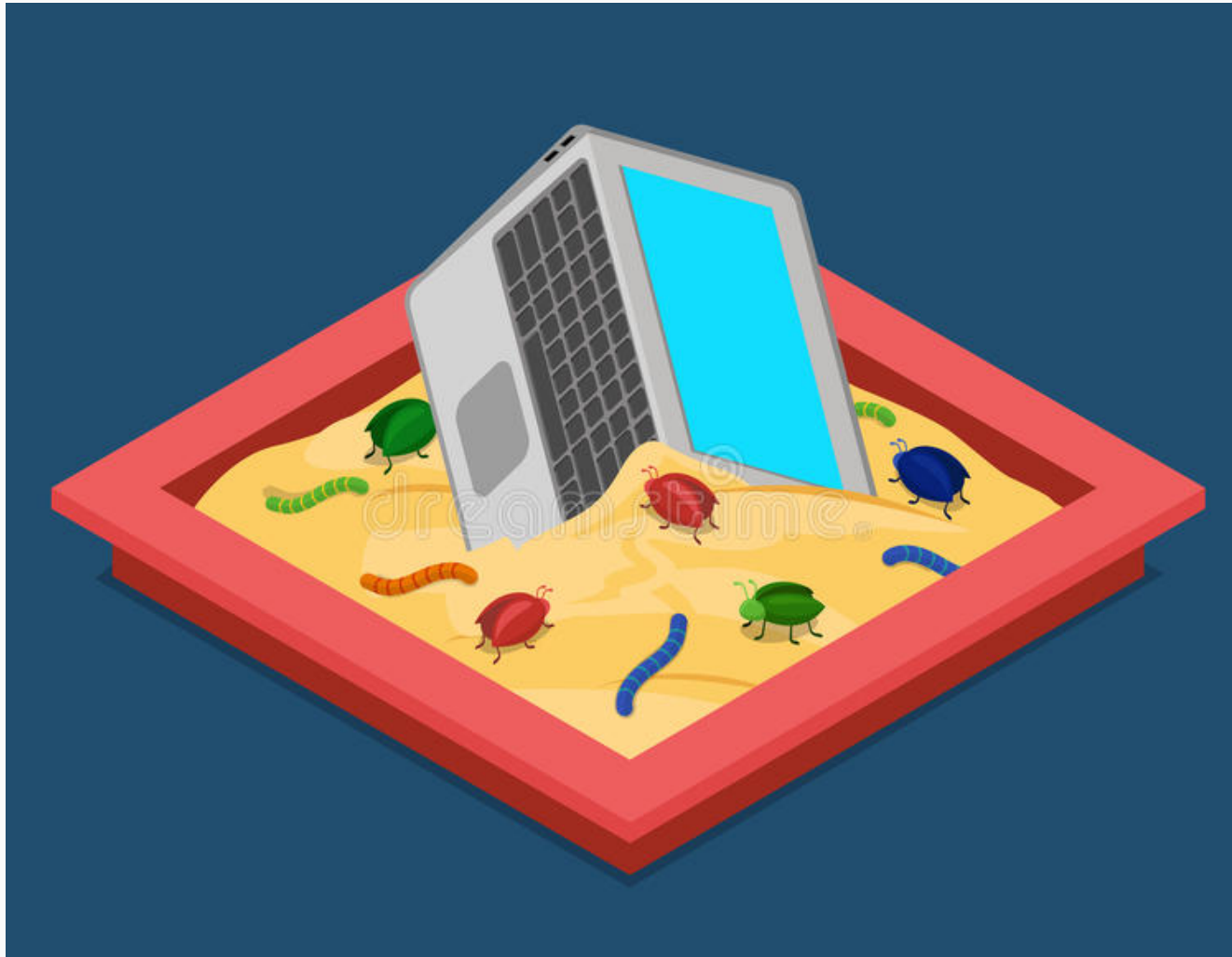
Oversikt

Virtualisering og sandboxing

Unikernels

Øving P5

Virtualisering vs sandboxing: isolering av system eller applikasjoner



- Full Virtualisering (sandboxing av et helt system)
- Eksempler: VirtualBox og VMWare



- OS-nivå virtualisering (applikasjon sandboxing)
- Eksempler:
 - Docker
 - Desktopapplikasjoner med Flatpak
 - Nettsider i en nettleser (?)

OS-nivå virtualisering, filsystem: chroot

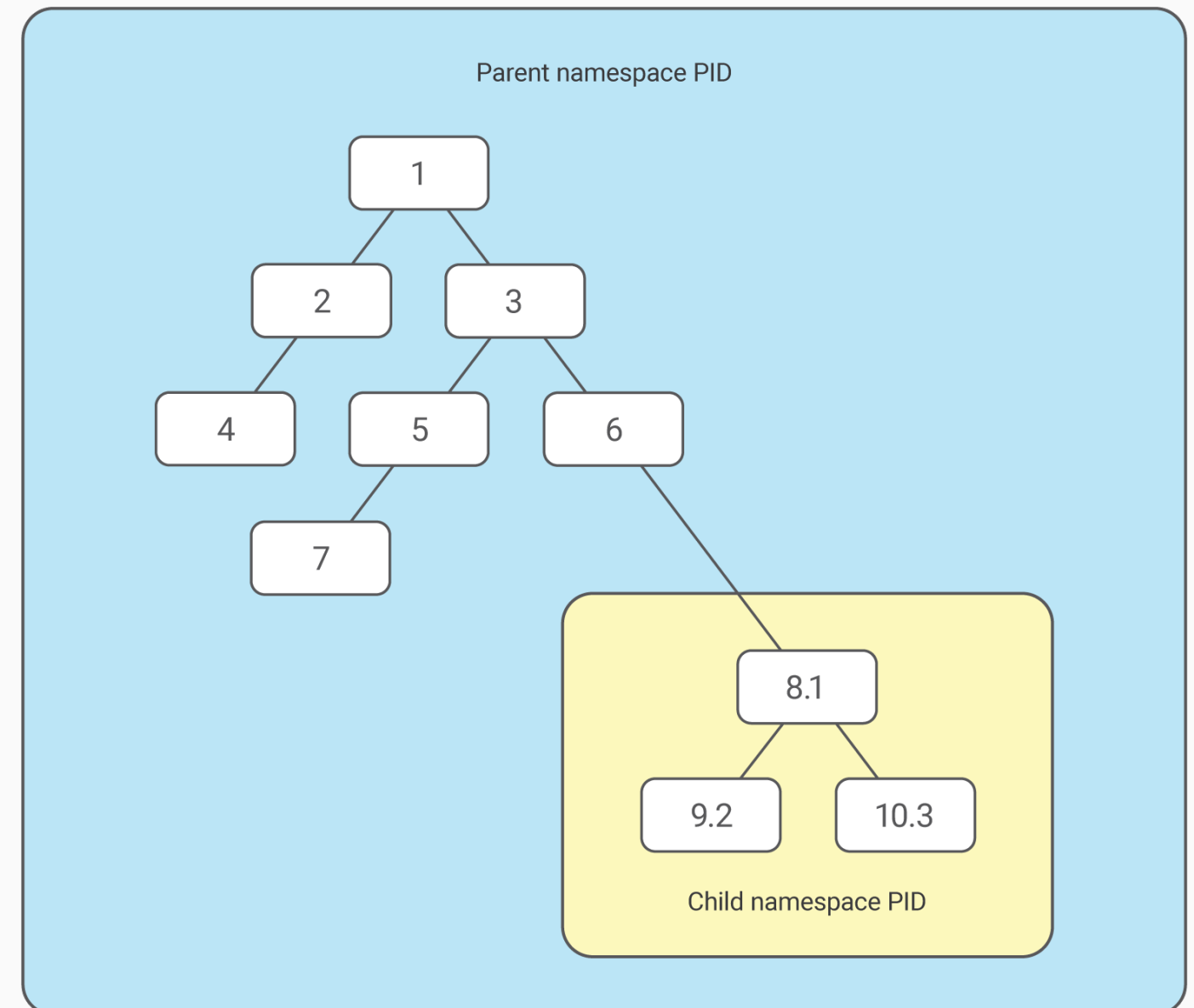
```
# Prepare new root folder:
mkdir new_root
cd new_root
mkdir bin lib64 usr usr/lib
cp /bin/bash bin/
cp /bin/ls bin/
ldd /bin/bash
# Copy all the output libraries, excluding linux-vdso.so.1,
# to one of the corresponding lib-folders in new_root/.
# For instance: cp /usr/lib/libreadline.so.8 usr/lib
ldd /bin/ls
# Copy all the output libraries, excluding linux-vdso.so.1,
# to one of the corresponding lib-folders in new_root/.

# Change root folder:
sudo chroot . /bin/bash # /bin/bash is run after changing root

# Test new root folder
ls / # Output: /bin /lib64 /usr
```

OS-nivå virtualisering, filsystem og prosesser: Linux namespaces

- chroot: isolerer kun filsystemet
- Linux Namespaces: isolerer på alle nivå gjennom ulike namespace typer, for eksempel:
 - *mnt*: som chroot men kan kombineres med for eksempel *pid*
 - *pid*: isolerer prosesser, for eksempel:
 - Prosess med id 7 kan se alle prosesser
 - Prosess 9 kan bare se 8, 9 og 10, men ser disse som 1, 2 og 3



OS-nivå virtualisering, prosesser: Linux namespaces

```
// Must be run as root

#include <iostream>
#include <sched.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

using namespace std;

int main() {
    char child_stack[1048576]; // Stack for the child process, 1024 * 1024 bytes

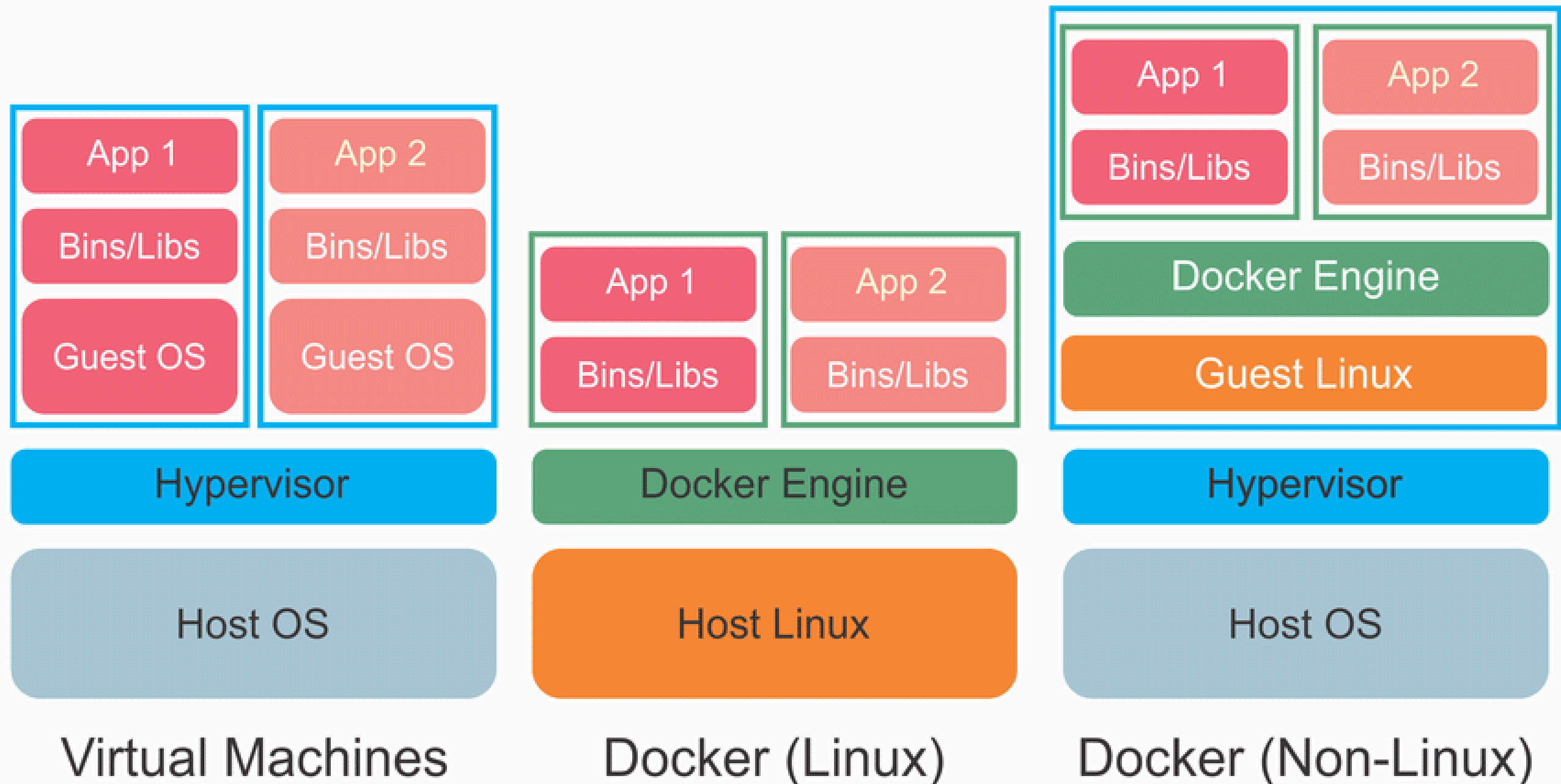
    // Create child process in new PID namespace
    pid_t pid = clone([](void *) {
        cout << "Child namespace PID: " << getpid() << endl; // Output: 1
        return 0;
    }, child_stack + 1048576 /* stack grows downwards */, CLONE_NEWPID, nullptr);

    cout << "Main PID : " << getpid() << endl; // Example output: 9838
    cout << "Parent namespace PID: " << pid << endl; // Example output: 9839

    waitpid(pid, nullptr, 0); // Wait for child process to exit
}
```

OS-nivå virtualisering: Docker

Bruker Linux namespaces og kan da gjenbruke vertsoverativsystemet:



OS-nivå virtualisering: Docker eksempel

- Kan bruke ferdige oppsatte docker images fra <https://hub.docker.com>
- For eksempel her startes `bash` i et docker image som inneholder siste Debian stable:
 - `docker run -ti --rm debian:latest /bin/bash`
 - `-ti`: for å kunne lese og skrive til `bash` i en terminal
 - `--rm`: slett docker containeren ved avslutning

OS-nivå virtualisering: bygge egne Docker images

Eksempel Dockerfile:

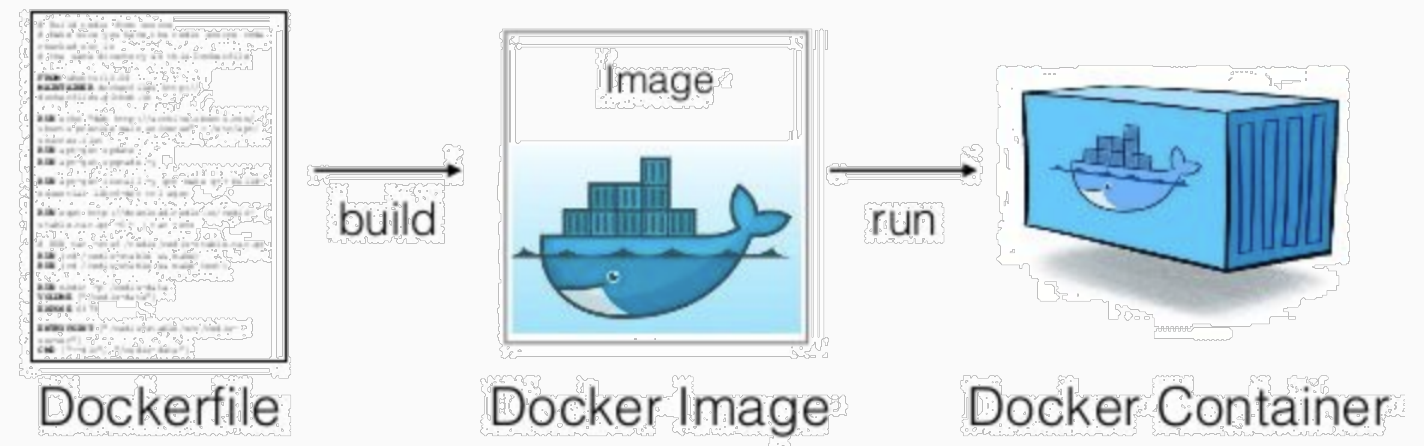
```
# Use Debian stable as base image
FROM debian:stable
# Upgrade packages
RUN apt-get -y update
# Install python
RUN apt-get -y install python3
```

Bygges med:

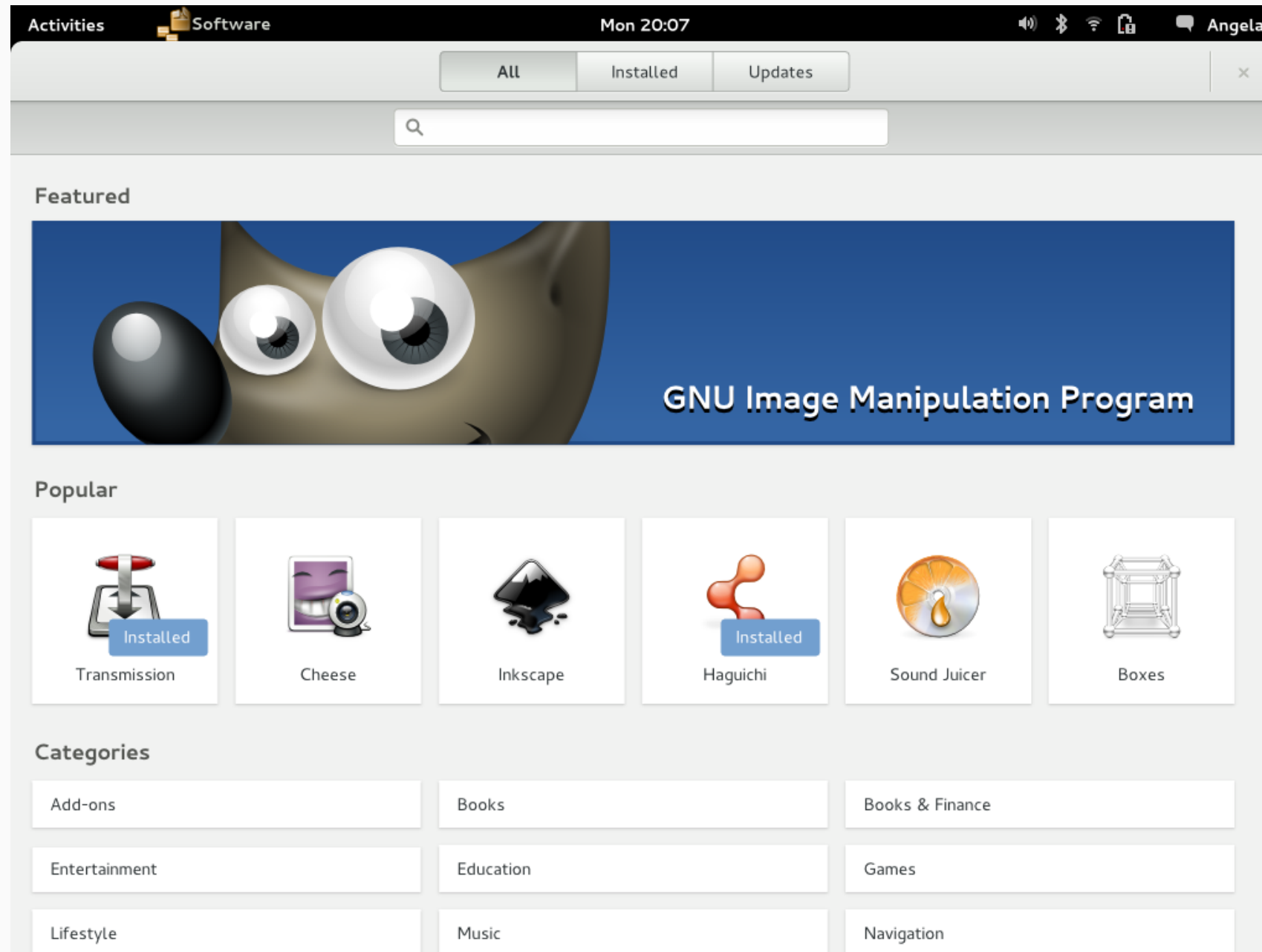
```
docker build -t python-image .
```

Kjøres med:

```
docker run --rm python-image python3 -c "print(\"Hello World\")"
# eller for å starte bash i kontaineren:
docker run -ti --rm python-image /bin/bash
```



OS-nivå virtualisering: Flatpak, pakkede desktop applikasjoner



Bruker Linux namespaces og kan kjøres i alle Linux distribusjoner

Oversikt

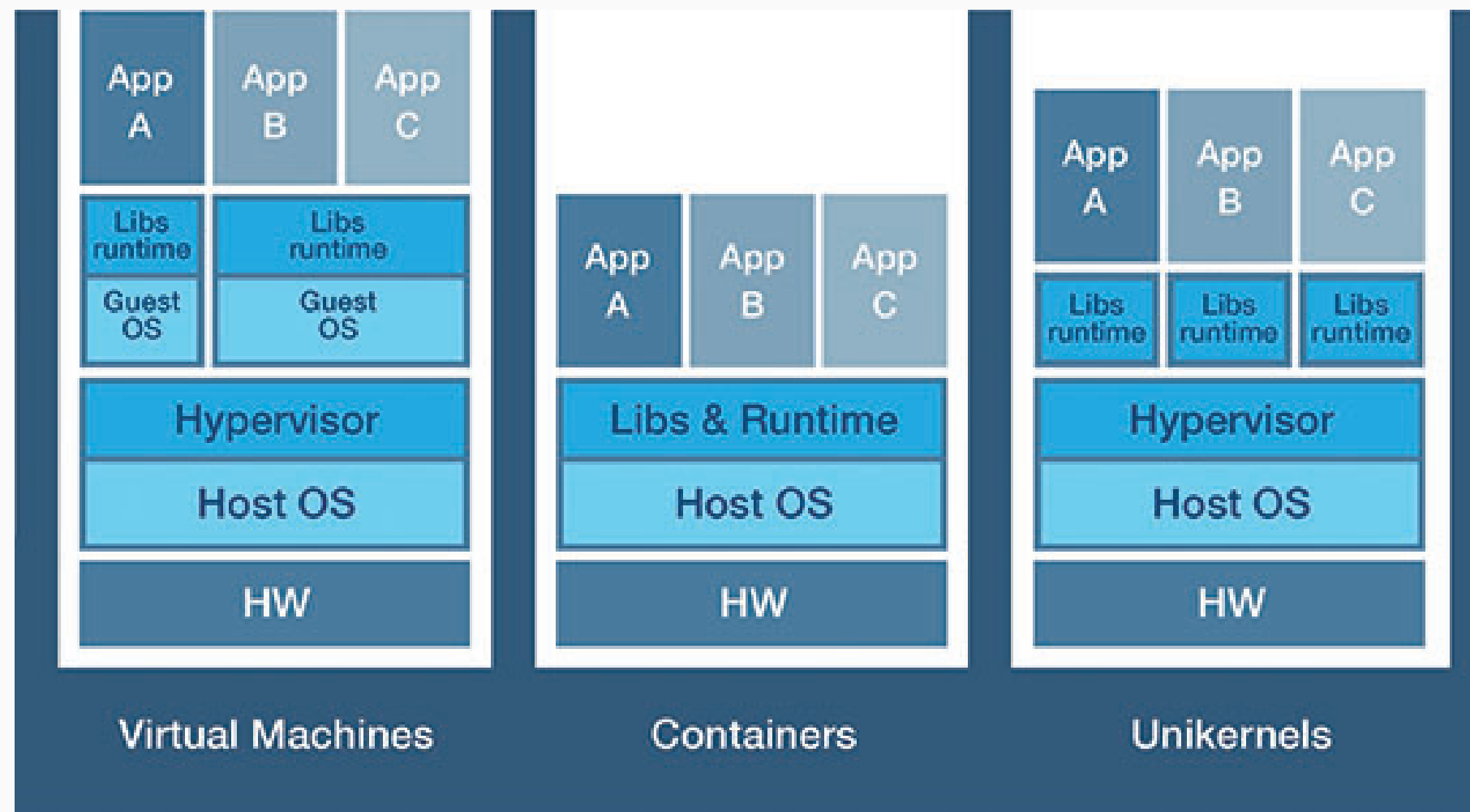
Virtualisering og sandboxing

Unikernels

Øving P5

Unikernels

- Unikernel applikasjoner bruker et minimalistisk operativsystem (unikernel) med kun de nødvendige bibliotekene for å kjøre applikasjonen (*library operating system*)
 - Mindre ressurskrevende og potensielt sikrere



Unikernels: kodeeksempel fra IncludeOS

```
#include <os>
#include <iostream>
#include <net/interfaces>

void Service::start() {
    // Get the IP stack thats already been automatically configured
    auto inet = net::Interfaces::get(0);
    // Setup a TCP echo server on port 7 (echo port)
    auto server = inet.tcp().listen(7);

    server.on_connect([] (auto conn) {
        // Log incomming connections on the console:
        std::cout << "Connection " << conn->to_string() << " established\n";
        // When data is received, echo back
        conn->on_read(1024, [conn](auto buf) {
            conn->write(buf);
        });
    });
}
```

Oversikt

Virtualisering og sandboxing

Unikernels

Øving P5

Øving P5, forberedelse: installasjon og oppsett av Docker

Instruksjonene er for den anbefalte Linux distribusjonen Manjaro, men andre distribusjoner (eller operativsystemer som støtter Docker) kan brukes.

Dokumentasjon: <https://wiki.archlinux.org/index.php/Docker>

```
sudo pacman -Syuu # Update system
sudo pacman -S docker # Install docker
sudo systemctl start docker # Start docker service
sudo systemctl enable docker # Enable docker service to be started on bootup

# To enable running docker as a regular user,
# add yourself ($USER) to docker group:
sudo usermod -aG docker $USER
# Finally, log out and back in to reevaluate group memberships
```

Se neste slide

Øving P5

- Lag en nettside som lar deg (kompile og) kjøre kildekode som blir skrevet inn
 - Resultatet skal vises på nettsiden
- Bruk Docker til å (kompile og) kjøre kildekoden trygt på serversiden
- Valgfritt programmeringsspråk (JavaScript og Python kan også brukes)
 - Gjelder både implementasjonen av web applikasjonen og programmeringsspråk som skal (kompileres og) kjøres i web applikasjonen

main.cpp:

```
#include <iostream>

using namespace std;

int main() {
    cout << "Hello World" << endl;
}
```

Compile and Run

Output:

```
Compiling and running main
Scanning dependencies of target main
[ 50%] Building CXX object main.cpp.o
[100%] Linking CXX executable main
[100%] Built target main
Hello World
main returned: 0
```